

MMA301 – MULTIPLATFORM MOBILE APP DEVELOPMENT

ASSIGNMENT 1

Profile & Activity App

Navigation • Components • State • Forms • Lists • Persistence

Nguyên tắc trọng tâm: Assignment không chỉ đánh giá sản phẩm chạy được. Sinh viên phải chứng minh chuỗi năng lực: hiểu yêu cầu → thiết kế có lý do → triển khai → kiểm thử → debug → giải thích → thích nghi khi yêu cầu thay đổi → sử dụng AI có trách nhiệm.

Thông tin	Nội dung
Assignment	Assignment 01 – React Native Foundations
Bối cảnh	Personal Profile & Activity App
CLO đánh giá chính	CLO1–CLO7
CLO củng cố	CLO9 – modeling/storing data ở mức AsyncStorage
Hình thức	Bài có tính điểm; demo/interview/verification
Điểm rubric	10.0 điểm / 8 tiêu chí
Ngôn ngữ	JavaScript (TypeScript không phải tiêu chí)
Nền tảng	React Native + Expo theo cấu hình lớp
Assessment category	Assignment category = 10% học phần cho 2 assignments; điểm từng bài báo cáo thang 10, trọng số theo cấu hình lớp

CÁCH SỬ DỤNG TÀI LIỆU

Tài liệu này không phải đáp án. Hãy dùng như bản đồ triển khai và checklist chuẩn bị đánh giá. Mỗi giai đoạn mô tả mục tiêu, quyết định kỹ thuật, bằng chứng và câu hỏi sinh viên phải tự trả lời.

Giai đoạn	Việc cần làm	Bằng chứng nên có
1. Understand	Đọc đề, lập RTM, xác định screen/workflow/state/data/constraint.	RTM, workflow sketch, data/state sketch.
2. Design	Thiết kế navigation, component tree, state ownership, persistence/integration.	Design decisions + sơ đồ.
3. Build	Triển khai theo milestone; commit theo chức năng; tránh one-shot code dump.	Git history + chức năng chạy từng phần.
4. Verify	Test normal/negative/edge; ghi defect và retest.	Test matrix + evidence.
5. Explain	Tự giải thích purpose/input/trigger/transform/output/failure path.	Interview rehearsal checklist.
6. Transfer	Phân tích và xử lý change request nhỏ không dựa vào code mẫu.	Impact analysis + patch/test.
7. Defend	Demo, interview, debug/modification và AI evidence.	Bộ hồ sơ đánh giá hoàn chỉnh.

0. THÔNG TIN ASSIGNMENT VÀ CHÍNH SÁCH ĐÁNH GIÁ

Mục	Yêu cầu
Tên bài	Assignment 01 – Profile & Activity App
Core scope	Home/Profile/Edit Profile/Activity or Interests/Settings
Navigation	Stack Navigator hoặc cấu hình tương đương theo lớp
State	useState + Context API
Form	Controlled input + Formik/Yup hoặc cơ chế validation theo lớp
List	FlatList hoặc SectionList cho dữ liệu collection
Persistence	AsyncStorage cho profile/theme/preference
UI	StyleSheet/Styled Components + Flexbox + reusable components
Project name	StudentName_ClassCode hoặc quy ước lớp

0.1 Sản phẩm cần nộp

- Source code React Native có thể cài dependency và chạy lại.
- README.md: overview, cách chạy, screen/workflow map, state/persistence strategy, limitations, AI usage.
- Requirement Traceability Matrix + Design Decision Record.
- Test matrix có Actual/Pass + tối thiểu 2 debug logs.
- AI usage log nếu có sử dụng AI ở mức đáng kể.
- Repository/Git history nếu lớp sử dụng Git.

0.2 Chính sách trung thực học thuật và AI

- Sinh viên chịu trách nhiệm với toàn bộ source code, cấu hình và tài nguyên nộp; phải giải thích được phần mình sử dụng.
- Được phép dùng slide, tài liệu chính thức, ví dụ đã học và AI assistant theo quy định của lớp.
- Nếu dùng AI ở mức đáng kể: ghi mục đích/prompt hoặc tham chiếu, phần đã áp dụng, phần đã sửa và cách kiểm chứng.
- Không nộp source trọn bộ do AI/người khác tạo mà không hiểu; code chạy nhưng không giải thích hoặc không sửa được yêu cầu nhỏ sẽ bị phản ánh ở rubric tương ứng.
- Không commit secret/API key thật vào repository hoặc file nộp.

0.3 Quy trình buổi đánh giá – mô hình khuyến nghị

Pha	Sinh viên thực hiện	Giảng viên quan sát/chấm
A. Identity & Run	Mở đúng bản nộp, chạy app bằng lệnh chuẩn, demo nhanh workflow chính.	Tính xác thực; môi trường; project chạy.
B. Requirement Demo	Demo từng requirement theo RTM, gồm cả failure/empty/edge state.	Problem Understanding + Implementation.
C. Code Interview	Mở file do giảng viên chỉ định và giải thích state/data/event flow.	Code Explanation + Design Reasoning.
D. Verification / Debug	Chạy test case hoặc phân tích một lỗi mới.	Testing + Debugging.
E. Transfer Task	Phân tích impact và thực hiện/mô tả một thay đổi nhỏ.	Modification / Transfer.
F. AI Evidence	Trình bày AI log nếu có và cách kiểm chứng output.	AI Usage & Engineering Practice.

Phạm vi: Assignment 1 là bài nền tảng. Ưu tiên workflow đúng, state rõ, list/persistence hoạt động và giải thích được trước khi thêm animation hoặc thư viện nâng cao.

1. TÓM TẮT YÊU CẦU VÀ PHẠM VI THỰC HIỆN

Xây dựng một ứng dụng Profile & Activity nhiều màn hình hoạt động như một workflow thống nhất, không phải tập hợp các màn hình demo rời rạc.

1.1 Màn hình và workflow bắt buộc

Screen	Trách nhiệm tối thiểu
HomeScreen	Welcome/summary; điều hướng tới Profile và Activity/Interests.
ProfileScreen	Hiển thị name/avatar/bio và dữ liệu đã lưu; nút Edit.
EditProfileScreen	Controlled form; validation; save/cancel; lỗi không ghi dữ liệu.
Activity/Interests Screen	FlatList/SectionList; ít nhất một interaction có state thay đổi.
SettingsScreen	Light/Dark hoặc preference app-wide; lưu persistent.

1.2 End-to-End Workflow

1. App khởi động → đọc dữ liệu profile/theme/preference đã lưu nếu có; first-run phải có default an toàn.
2. Profile → Edit → nhập dữ liệu → validation → save → state/shared UI cập nhật đúng mà không cần restart.
3. Activity/Interests → render collection bằng FlatList/SectionList → interaction cập nhật state/list hợp lệ.
4. Settings → đổi theme/preference → áp dụng app-wide → restart app vẫn giữ đúng theo design.
5. Missing/corrupted local data không làm app crash; có fallback hợp lý.

1.3 Requirement Traceability Matrix tối thiểu

ID	Yêu cầu	Bằng chứng UI/code	Test tối thiểu
R01	Expo/React Native project chạy đúng quy ước	package.json/app config	cold start
R02	Navigation giữa ≥5 khu vực/screen	navigation config	mở từng screen/back
R03	Profile hiển thị dữ liệu	ProfileScreen	default + persisted
R04	Edit Profile + validation	form/state	valid/invalid/cancel
R05	Theme/preference shared bằng Context	ThemeContext	toggle ở nhiều screen
R06	FlatList/SectionList	Activity/Interests	non-empty + empty
R07	Persistence bằng AsyncStorage	storage helper/useEffect	first-run + restart
R08	Reusable components	ProfileCard/ThemeToggle/etc.	render/reuse
R09	Responsive Flexbox/style	screen/components	portrait/sizes cơ bản
R10	Error/fallback state	storage/validation	invalid local data/no item

1.4 Phạm vi không bắt buộc

- Không bắt buộc Firebase/backend/API thật.
- Không bắt buộc authentication production.
- Không bắt buộc Redux nếu Context đủ cho scope.
- Không bắt buộc TypeScript.
- Không chấm cao chỉ vì thêm nhiều thư viện/animation ngoài core evidence.

2. HƯỚNG DẪN PHÂN TÍCH YÊU CẦU – PROBLEM UNDERSTANDING

Screen/area	Input / state	Process	Output / failure
Profile	profile state	read/render	profile card / default
Edit Profile	name,bio,avatar/path	validate → save/cancel	updated profile / validation error
Activity/Interests	collection + item action	render/update/filter if used	list/empty state
Settings	theme/preference	toggle → shared state → persist	app-wide change / fallback
Startup	AsyncStorage values	parse/hydrate/default	restored state / safe default

2.2 Worksheet – điền trước khi code

- State nào local? state nào shared? state nào persistent?
- Profile model gồm field nào? validation rule nào?
- List dùng key nào và vì sao?
- Khi storage chưa có dữ liệu hoặc JSON lỗi thì app làm gì?
- Context đặt ở đâu để tránh share state quá rộng?
- Save profile cập nhật UI nào ngay lập tức?
- Theme/preference hydrate trước/sau render thế nào để tránh nháy sai giao diện?

3. HƯỚNG DẪN THIẾT KẾ GIẢI PHÁP – SOLUTION DESIGN & REASONING

Cấu trúc dưới đây là gợi ý, không phải đáp án bắt buộc. Điềm đến từ quyết định phù hợp và giải thích được trade-off.

App → Navigation → Screens → Reusable Components
Shared Theme/Profile Context → AsyncStorage helper
Activity/Interests list → item component / optional custom hook

3.1 State ownership map

State	Ví dụ	Nên ở đâu?	Lý do
Local UI	input focus, modal/dropdown open	screen/component	không cần share
Form	name/bio/errors	EditProfile	gắn vòng đời form
Shared profile	profile object	Context hoặc parent owner	nhiều screen cần đọc
Theme/preference	theme mode	Context	app-wide
Persistent	profile/theme	AsyncStorage + hydrate	giữ qua restart
List	activities/interests	screen/context tùy scope	đủ gần consumer

3.2 Design Decision Record – tối thiểu 5 quyết định

Quyết định	Lựa chọn của tôi	Lý do	Impact nếu đổi requirement
Navigation			
State ownership			
Persistence/hydration			
List item structure			
Folder/component reuse			

4. HƯỚNG DẪN TRIỂN KHAI THEO MILESTONE – IMPLEMENTATION

Milestone	Gate đạt	Cách kiểm tra	Evidence
M1	Project chạy + navigation skeleton	npm/expo start + mở 5 screen	terminal + UI
M2	UI/components + theme context	theme app-wide	component/state sketch
M3	Profile read/edit + validation	valid/invalid/cancel	form tests
M4	FlatList/SectionList	non-empty/empty + interaction	list evidence
M5	AsyncStorage hydrate/save	first-run + restart	storage evidence
M6	Integration/refactor	workflow xuyên screen	Git/evidence
M7	Verify/debug/release candidate	smoke test + 2 debug logs	README + test matrix

Không có source hoàn chỉnh: Milestone chỉ là checkpoint. Sinh viên tự viết/điều chỉnh code và chịu trách nhiệm với design decision.

5. HƯỚNG DẪN KIỂM THỬ – TESTING & VERIFICATION

#	Nhóm	Case	Thao tác	Kết quả mong đợi
1	Startup	First run	Không có AsyncStorage	app dùng default, không crash
2	Profile	Read	mở Profile	đúng dữ liệu
3	Edit	Invalid	name rỗng	không save; có lỗi
4	Edit	Valid	đổi name/bio	Profile cập nhật
5	Edit	Cancel	thay đổi rồi cancel	không ghi
6	Theme	Toggle	đổi light/dark	app-wide cập nhật
7	Theme	Persistence	restart	theme còn đúng
8	List	Normal	có items	FlatList/SectionList đúng
9	List	Empty	mảng rỗng	empty state
10	List	Interaction	chọn/mark/filter theo design	state/UI đúng
11	Storage	Corrupt/missing	invalid local value	fallback an toàn
12	Navigation	All screens	open/back	không crash/mất state ngoài design

6. HƯỚNG DẪN DEBUG – DEBUGGING & PROBLEM SOLVING

Quy trình 5 bước: (1) mô tả triệu chứng quan sát được; (2) khoanh vùng screen/component/state/service; (3) kiểm tra input/state; (4) kiểm tra điểm biến đổi/async call; (5) sửa nhỏ nhất → chạy lại case → kiểm tra regression.

Triệu chứng	Điểm kiểm tra đầu tiên	Câu hỏi dẫn đường
Theme không cập nhật	Context provider/value	Provider có wrap đúng? state có đổi?
Profile save nhưng UI cũ	shared profile/state	save vào state nào? consumer đọc state nào?
Reload mất dữ liệu	AsyncStorage hydrate/save	await đúng? key giống nhau? parse fallback?
FlatList không re-render	data reference/key	array mới? keyExtractor ổn?
Form lỗi cũ còn	form/reset lifecycle	reset khi screen focus/submit/cancel?

6.2 Debug Log – tối thiểu 2 lỗi thật

Lỗi	Giả thuyết	Cách kiểm tra	Root cause	Fix	Retest

Trong buổi chấm: Nếu giảng viên đưa ra bug mới, hãy nói thành tiếng: triệu chứng → giả thuyết → nơi kiểm tra → bằng chứng → sửa → retest.

7. HƯỚNG DẪN CHUẨN BỊ GIẢI THÍCH CODE – CODE EXPLANATION

Sinh viên phải mở được code của mình và giải thích theo luồng thực thi; không nên chỉ trả lời định nghĩa thuộc lòng.

Tầng	Cách giải thích
1. Purpose	Screen/component/hook/service này giải quyết trách nhiệm gì?
2. Input	Props/state/data/parameter nào đi vào?
3. Trigger	Event/useEffect/navigation/user action nào kích hoạt?

Tầng	Cách giải thích
4. Transform	Logic nào validate/filter/map/call API/update state?
5. Output	State/UI/navigation nào thay đổi? Failure path ra sao?

7.2 Chủ đề phải giải thích được

- Navigation stack và trách nhiệm từng screen.
- Props/state/context khác nhau ở project này.
- Controlled form + validation + save/cancel flow.
- Context provider/value và consumer re-render.
- FlatList/SectionList: data, renderItem, keyExtractor, empty state.
- AsyncStorage: key, serialization, hydrate/save và failure handling.
- Vì sao không mutate object/array trực tiếp.
- Folder structure và reusable components.

7.3 “Mở file và chỉ code” – rehearsal checklist

Yêu cầu rehearsal	Tôi làm được trong ≤30 giây?
Mở navigation config và chỉ route/screen	☐
Mở Theme/Profile Context và chỉ provider/value	☐
Mở EditProfile save/validation logic	☐
Mở AsyncStorage helper/hydration	☐
Mở FlatList renderItem/keyExtractor	☐
Mở component tái sử dụng	☐

8. HƯỚNG DẪN XỬ LÝ YÊU CẦU THAY ĐỔI – MODIFICATION / TRANSFER TASK

Mục tiêu: chứng minh sinh viên không chỉ nhớ code hiện tại mà có thể chuyển kiến thức sang requirement mới. Trước khi code, hãy phân tích impact.

8.1 Bài luyện mức cơ bản

- Thêm field phone hoặc location cho Profile.
- Thêm Reset form.
- Thêm filter nhỏ cho Activity/Interests.
- Đổi validation rule của name/bio.

8.2 Bài luyện mức trung bình

- Thêm Favorites/mark item và persist.
- Thêm system theme option ngoài Light/Dark.
- Tách storage helper khỏi screen nếu đang viết trực tiếp.
- Thêm sort hoặc section grouping cho list.

8.3 Bài luyện nâng cao

- Thêm simple mock authentication trước Profile.
- Thay AsyncStorage profile bằng mock API/service nhưng giữ UI behavior.
- Thêm offline draft cho Edit Profile.

8.4 Impact Analysis Worksheet

Change request	Screen/component	State/data	Service/storage	UI/permission	Tests phải sửa/thêm

Change request	Screen/component	State/data	Service/storage	UI/permission	Tests phải sửa/thêm

9. HƯỚNG DẪN SỬ DỤNG AI CÓ KIỂM SOÁT – AI USAGE & ENGINEERING PRACTICE

9.1 AI có thể hỗ trợ

- Giải thích concept hoặc đoạn code do sinh viên đang làm.
- Gợi ý component/state/data structure hoặc refactoring.
- Đưa giả thuyết debug và checklist kiểm tra.
- Sinh test case/edge case để sinh viên chọn và kiểm chứng.
- Review naming, UX/accessibility, validation, README hoặc design alternatives.

9.2 AI không được thay thế năng lực

- Không dùng AI để tạo toàn bộ assignment rồi nộp như sản phẩm của mình.
- Không coi output AI là đúng nếu chưa chạy/test/đối chiếu requirement.
- Không dùng AI để che giấu việc không giải thích được state/event/data flow.
- Trong buổi đánh giá trực tiếp, lecturer có thể yêu cầu modification/debug không dùng AI.

9.3 Quy trình AI → Verify → Adopt

6. Ask – đặt câu hỏi cụ thể.
7. Inspect – đọc output và xác định giả định/rủi ro.
8. Adapt – chỉnh cho project của mình.
9. Run – chạy đúng test case.
10. Explain – tự giải thích được logic và side effect.
11. Record – ghi prompt/reference + phần đã đổi + cách kiểm chứng.

9.4 AI Usage Log tối thiểu

#	Task / Prompt / Ref	Mục đích	Output dùng?	Tôi đã sửa gì?	Tôi kiểm chứng thế nào?

10. RUBRIC ĐÁNH GIÁ – 10.0 ĐIỂM

Rubric đánh giá năng lực thực chất, không chỉ đánh giá việc sản phẩm “có chạy”. Implementation tối đa 2.0/10; 8.0 điểm còn lại đến từ hiểu bài toán, thiết kế, kiểm chứng, debug, giải thích, transfer và AI engineering practice.

#	Tiêu chí	Điểm tối đa	Bảng chứng chính
1	Problem Understanding	1.0	RTM + workflow/state/persistence constraints.
2	Solution Design & Reasoning	1.5	Navigation/component/state ownership + decision record.
3	Implementation	2.0	5 screen workflow + form/list/context/persistence.
4	Testing & Verification	1.5	Test matrix normal/negative/edge + actual result.
5	Debugging & Problem Solving	1.5	2 debug logs + live reasoning.
6	Code Explanation	1.0	Mở code và giải thích navigation/state/list/storage flow.
7	Modification / Transfer Task	1.0	Change request nhỏ + impact + retest.
8	AI Usage & Engineering Practice	0.5	AI log + verify/adapt/explain.

10.1 Mức đánh giá chi tiết

Tiêu chí	Excellent / Tối đa	Good / Khá	Fair / Trung bình	Poor / Chưa đạt
Problem Understanding – 1.0	0.9–1.0: Nêu đúng scope, workflow, constraint, input/output; RTM khớp yêu cầu và test.	0.7–0.8: Hiểu phần lớn; thiếu chi tiết nhỏ nhưng không sai core requirement.	0.4–0.6: Hiểu rời rạc; bỏ sót constraint hoặc không rõ input/output/failure state.	0–0.3: Không mô tả được bài toán; nhầm core requirement hoặc chỉ dựa vào code có sẵn.
Solution Design & Reasoning – 1.5	1.35–1.50: Navigation/component/state/data thiết kế hợp lý; nêu trade-off và impact khi đổi yêu cầu.	1.05–1.30: Thiết kế hợp lý, giải thích được phần chính; một số quyết định còn theo thói quen.	0.60–1.00: Cấu trúc chạy được nhưng ownership/coupling chưa rõ; khó giải thích decision.	0–0.55: Thiết kế chấp vá; logic rời rạc; không biết vì sao đặt state/data/logic như vậy.
Implementation – 2.0	1.80–2.00: Core workflow đầy đủ, tích hợp ổn định, state/data update đúng, failure state hợp lý.	1.40–1.75: Core functions hầu hết chạy; có lỗi nhỏ hoặc chất lượng chưa đều.	0.80–1.35: Chạy một phần; thiếu/mất ổn định ở workflow chính; code khó duy trì.	0–0.75: Project không chạy hoặc thiếu nhiều core requirements; không làm chủ source.
Testing & Verification – 1.5	1.35–1.50: Test matrix đủ normal/negative/edge; expected/actual rõ; bug fix có retest.	1.05–1.30: Test core functions và một số edge case; evidence tương đối rõ.	0.60–1.00: Test chủ yếu happy path; thiếu edge/actual evidence.	0–0.55: Hầu như không có test có cấu trúc; chỉ demo ngẫu nhiên.
Debugging & Problem Solving – 1.5	1.35–1.50: Khoanh vùng có hệ thống, dùng log/DevTools/debugger hợp lý, tìm root cause và retest.	1.05–1.30: Sửa được lỗi phổ biến, giải thích nguyên nhân tương đối rõ.	0.60–1.00: Thử-sai nhiều; cần gợi ý; sửa được nhưng chưa rõ root cause.	0–0.55: Không biết bắt đầu debug; phụ thuộc người khác/AI để sửa.
Code Explanation – 1.0	0.9–1.0: Mở đúng file và giải thích purpose-input-trigger-transform-output/failure path; hiểu state/data flow.	0.7–0.8: Giải thích được phần lớn code mình dùng; còn một số chi tiết chưa sâu.	0.4–0.6: Giải thích cú pháp cơ bản nhưng không theo được luồng dữ liệu/thực thi.	0–0.3: Không giải thích được code quan trọng hoặc câu trả lời mâu thuẫn với source.
Modification / Transfer Task – 1.0	0.9–1.0: Phân tích impact, sửa change request đúng, kiểm thử lại và giải thích thay đổi.	0.7–0.8: Thực hiện được thay đổi với ít gợi ý; test cơ bản.	0.4–0.6: Cần nhiều gợi ý; sửa được một phần hoặc gây regression.	0–0.3: Không thể xử lý thay đổi nhỏ nếu không có mẫu sẵn.
AI Usage & Engineering Practice – 0.5	0.45–0.50: Dùng AI minh bạch, có mục tiêu, tự kiểm chứng/chỉnh sửa, giải thích được kết quả.	0.35–0.40: Dùng AI hợp lệ và có kiểm chứng nhưng log/giải trình chưa đầy đủ.	0.20–0.30: Có dùng AI nhưng kiểm chứng hạn chế; phụ thuộc gợi ý khá nhiều.	0–0.15: Không minh bạch hoặc phụ thuộc source/AI trọn bộ mà không hiểu.

10.2 Scorecard sinh viên tự chấm trước đánh giá

Tiêu chí	Max	Tự chấm	Evidence tôi sẽ mở
Problem Understanding	1.0		
Solution Design & Reasoning	1.5		
Implementation	2.0		
Testing & Verification	1.5		
Debugging & Problem Solving	1.5		
Code Explanation	1.0		
Modification / Transfer Task	1.0		
AI Usage & Engineering Practice	0.5		
TỔNG	10.0		

11. CHECKLIST TRƯỚC KHI NỘP VÀ TRƯỚC BUỔI ĐÁNH GIÁ

11.1 Checklist yêu cầu bắt buộc

- ☐ App chạy; có ≥5 khu vực/screen theo scope.
- ☐ Navigation hoạt động.
- ☐ Profile read/edit + validation.
- ☐ Theme/preference app-wide.
- ☐ FlatList/SectionList + empty state.

- ☐ AsyncStorage restore/save.

11.2 Checklist chất lượng kỹ thuật

- ☐ Không có runtime/console error nghiêm trọng.
- ☐ State không mutate trực tiếp.
- ☐ Component/state tách hợp lý.
- ☐ Storage có fallback first-run/corrupt data.
- ☐ README chạy được từ môi trường sạch.

11.3 Checklist sẵn sàng phỏng vấn

- ☐ Giải thích được 2 flow: Edit Profile và Theme/Persistence.
- ☐ Mở đúng file Context/storage/list trong 30 giây.
- ☐ Phân tích được một bug state/persistence.
- ☐ Đã luyện ít nhất một modification task.

11.4 Checklist hồ sơ đánh giá

- ☐ Source/zip hoặc repo đúng bản.
- ☐ README final.
- ☐ RTM + Design Decision Record.
- ☐ Test matrix final.
- ☐ 2 debug logs.
- ☐ AI log nếu có.

11.5 Kế hoạch chuẩn bị 45–60 phút trước đánh giá

Thời gian	Việc làm
0–10 phút	Mở/clone đúng bản nộ; cài dependency; chạy app; xác nhận đúng version.
10–25 phút	Chạy smoke test toàn bộ core workflow + negative/edge cases chính.
25–35 phút	Mở RTM/design/state/data sketch; luyện giải thích 2–3 luồng chính.
35–45 phút	Luyện 1 debug case và 1 modification case không dùng AI.
45–60 phút	Kiểm tra README/test matrix/debug log/AI log/evidence; chuẩn bị demo trạng thái ổn định.

12. PHỤ LỤC – CÂU HỎI INTERVIEW / TỰ LUYỆN

12.1 React Native fundamentals

- Component/props/state khác nhau thế nào?
- Điều gì làm component re-render?
- Vì sao FlatList cần keyExtractor?
- Vì sao không mutate state?

12.2 State & persistence

- State nào local/shared/persistent?
- Khi restart app, quá trình hydrate diễn ra ở đâu?
- Nếu AsyncStorage trả null/JSON lỗi thì xử lý thế nào?

12.3 Design reasoning

- Vì sao chọn Context cho theme/profile?
- Nếu thêm một screen cần profile, cần thay đổi gì?
- Khi nào custom hook hợp lý?

12.4 Debug / transfer

- Profile không cập nhật sau Save: kiểm tra gì?
- Thêm phone field: file/state/storage/test nào bị ảnh hưởng?

12.5 AI usage

- Bạn dùng AI vào phần nào?
- Bạn kiểm chứng output bằng test nào?
- Bạn đã sửa gì so với gợi ý AI?

12.6 Final self-declaration

Sinh viên tự xác nhận: “Tôi đã tự kiểm tra assignment, có thể giải thích các phần chính, có thể phân tích/sửa một thay đổi nhỏ và chịu trách nhiệm với mọi code, tài nguyên và AI output tôi sử dụng.”